

九齿 .skill 创新挑战 (T3-1-1) 最终报告

- 参赛者：楚凌志
- Skill 名称：ninetoothed-operator-dev
- 交付物：InfiniTensor/ninetoothed PR, fork noCharger/ninetoothed, 分支 2026-spring-noCharger-T3-1-1, commit 5ee2963。Skill 包位于 skills/competition/ninetoothed-operator-dev/
- 配套评测 harness：noCharger/ninetoothed-skill-eval-harness, 分支 core-harness-v2, commit 156ac16。仅用于复现实验，不属于交付物

核心结果：在同任务、同 agent、同模型、同评分器的受控 A/B 下，本 skill 把 GLM-5.2 后端的算子实现通过率从 33% 提升到 100%；换到能力更强的 DeepSeek-v4-pro 后端，算子组仍净增 7 分并多攻克一道 holdout 难题。两个基座共 46 集有效对比，零确证负迁移。过程级分析进一步显示 skill 消除的核心问题是假自信：no_skill 组自测全绿却有过半题目通不过独立 oracle，with-skill 组的自测结论与独立判定在 GLM 组完全一致。全部 with-skill episode 均有 trajectory 级证据证明 skill 被真实调用；修复后的 softmax 算子在 RTX 5090 上实测达到 PyTorch 基线的 1.43 倍。

与中期报告的衔接

中期报告交付了 v0 skill 包和评测设计：六子项 rubric、三门检查、24 题自测任务集，以及“同条件 A/B + 三条达标线”的自测计划。终期把这套设计完整跑成了实测闭环——两个基座模型各 24 个真实 episode，自测计划的记录模板和达标线在第七节逐项兑现。相对中期有三处演进：评测代码划分成“skill 仓库内的静态自测材料”与“独立 harness 仓库的运行自动化”两层，skill 包本体保持零依赖可独立安装；Stage 3 自进化从两条路线收敛为轻量 MOO 一条，bandit + memory + distill 的重控制器路线经实测无增益后放弃并从代码库移除，失效的完整归因见第八节；评测可信度方面在中期已有的防作弊机制之上，新增了 trajectory 级的 skill 使用验证，并靠它发现了一处基线数据缺陷，详见第六节。

一、Skill 目标、设计原则与包结构

1.1 目标

让通用 coding agent 在 NineToothed DSL 算子开发这个垂直领域达到熟练贡献者水平：正确写出 arrangement 与 application、用矩阵测试自证正确性、做性能归类、诊断失败并给出最小修复。对 DSL 表达能力之外的任务，走声明式回退，而不是硬编或伪装成 DSL 实现。

1.2 设计原则

三条原则中期确立，终期全部落成可检验的机制。

第一，人工整理优于模型自生成。references 全部人工整理，并对照 `ninetoothed==0.25.0` 源码逐条核验。终期进一步给每条承重结论打上证据标签：[E] 表示在本项目的 GPU episode 中端到端验证过，[S] 表示对照源码核验但未执行，[I] 表示由相邻模式推断、需复核。agent 可以按条校准信任程度。

第二，按需渐进加载。入口 SKILL.md 只有 176 行，六份家族 playbook 共 867 行，只在工作流走到对应步骤时才加载。

第三，脚本承载确定性。凡是可判定的事情——正确性矩阵、越界检查、生成源码检视、计时、失败归因——都交给脚本完成，不依赖模型自觉。所有脚本无 eval、无 exec、无网络调用。

1.3 包结构

```
ninetoothed-operator-dev/
SKILL.md                # 触发 / 工作流 / 终止规则 / 产出清单 / 约束
references/              # 6 份 playbook, 含 [E]/[S]/[I] 证据标签
  operator-taxonomy.md   # 家族分类 + DSL 可表达性预筛
  elementwise.md         # reduction.md layout.md perf-diag.md common-errors.md
scripts/                # 7 个工具
  gen_pytorch_oracle.py  # PyTorch 参考实现 + pytest 脚手架
  run_correctness_matrix.py # shape×dtype×layout 矩阵, MERE/MARE 双指标
  inspect_generated_source.py # 读生成源码; --contract 做语义契约检查
  bench_compare.py       # CUDA-event 计时 + Roofline 判类
  debug_arrangement.py   # 编译前验证 arrangement
  failure_classifier.py   # 失败归因
  aot_build_smoke.sh     # AOT 构建冒烟
examples/               # 4 个完整已解任务, 每家族一个
tests/
  self_test_tasks.md     # 4 个自测任务定义
  verifier_spec.md       # 六子项评分规则 + 三门映射
evaluation/             # 自测材料, 不属于可安装 skill 本体
  proxy_tasks/           # 24 题离线集, 参考解仅用于打分
  skill_eval/            # robust bench + reward-hacking 防护
REFERENCE.md            # 引用与来源披露
```

`evaluation/` 是静态自测材料，随 skill 仓库交付以保证评测工作可审；驱动 agent 的运行自动化在配套 harness 仓库，harness 从 skill 的检出路径读取这些材料，自己不带副本。安装 skill 不需要 harness。

二、核心工作流

SKILL.md 定义九步流水线，每步只加载所需 reference：

步骤	动作	工具 / Reference
1. Spec	提取输入输出、shape、dtype、broadcast、stride、边界	—
2. Classify	定算子家族，同时做 DSL 可表达性预筛	operator-taxonomy.md
3. 找先例	搜仓库相似 arrangement，优先复用	—
4. Arrange + apply	按仓库风格写 arrangement 与 application	references/<family>.md
5. Debug arrangement	编译前确认无越界、tile 映射正确	debug_arrangement.py
6. Oracle + 正确性	生成 PyTorch 参考，跑 shapexdtypexlayout 矩阵	gen_pytorch_oracle.py 等
7. 检视生成源码	读 tile 与并行参数；reduction/scatter 类加查语义契约	inspect_generated_source.py
8. Benchmark + Roofline	对比 PyTorch 基线，判 compute/memory-bound	bench_compare.py
9. 失败诊断	症状 → 根因 → 最小修复 → 复验	common-errors.md

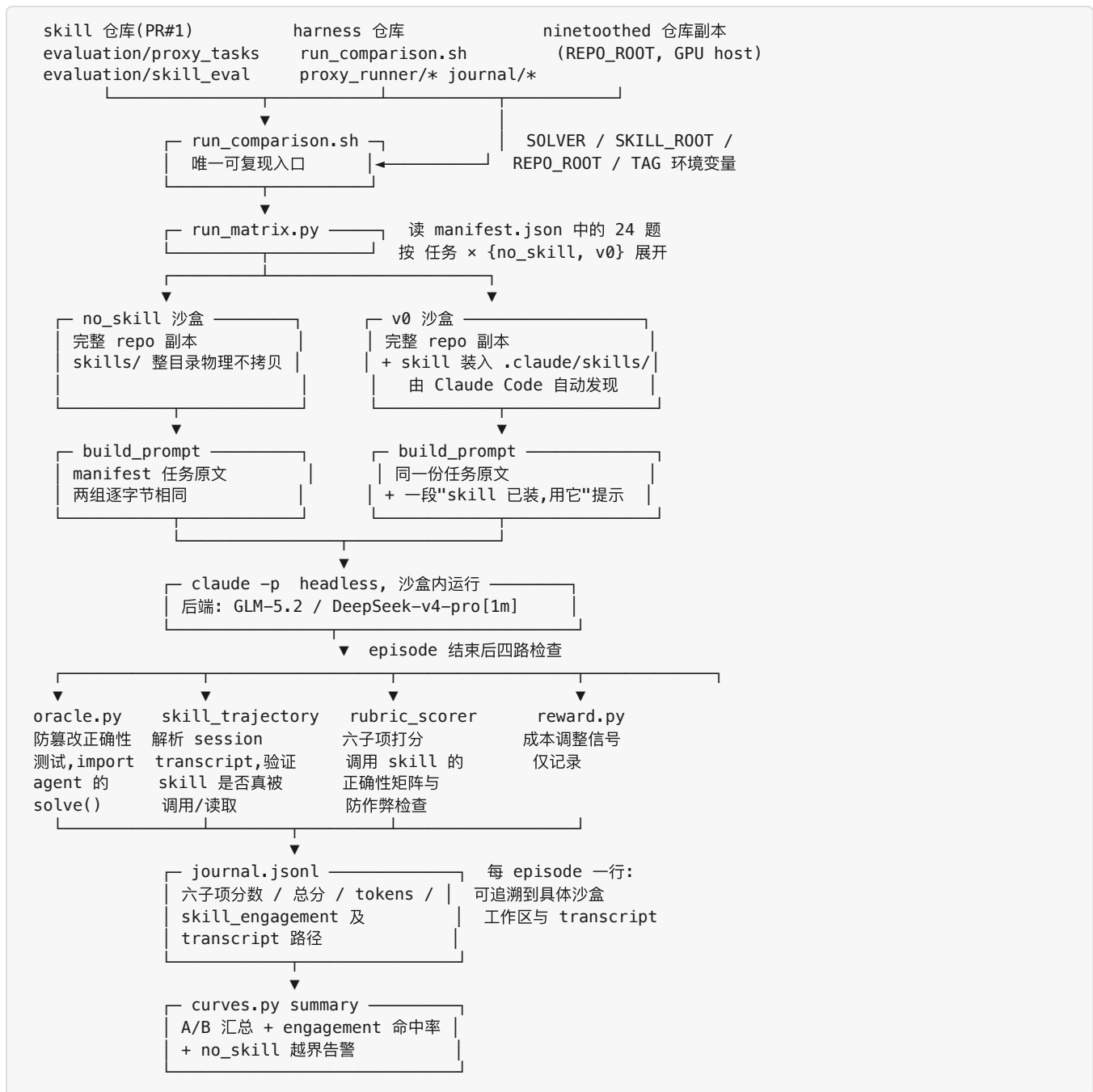
终止规则是本 skill 区别于提示词堆砌的核心：

- **先归因再重试**。任何失败先过 `failure_classifier.py` 做确定性分类。`code_error` 修 kernel；`guidance_error` 说明 skill 文本有缺口，记录后停止空转；`dsl_limit` 是终态——任务落在已知不可表达集合内，比如数据依赖索引、many-to-one scatter、值依赖控制流、动态输出 shape、跨 tile 通信，此时两侧都无修复目标，直接停止迭代；`unknown` 做根因分析后回填 `common-errors.md`。
- **声明式回退，绝不静默**。DSL 之外的任务在 wrapper 里实现 PyTorch 回退，并打上 `# ninetoothed-fallback: <feature> - <reason>` 标记。合法性门据此区分声明的回退与伪装的回退：前者 completion 低分但不违规，后者双重违规。
- **数值容差红线**。fp16/bf16 只差容差时，先做 dtype 上转修复再测一次，禁止放宽容差凑通过。
- **硬上限**。同根因三次 `code_error` 后强制回退，单算子十次迭代封顶。

三、评测链路与运行环境

3.1 跑法全景

每个 episode 都走同一条链路：



保证 delta 可信的机制：

机制	防什么	位置
防篡改 oracle	agent 改测试凑通过	harness 独立生成测试，直接 import agent 的 <code>solve()</code>
diff 限定打分	把仓库原有代码算成 agent 产出	episode 前快照基线清单，只对真实 diff 打分
合法性门	绕过 NineToothed 直接调 <code>torch.*</code>	AST 扫描，命中则 completion 封顶 1 分
语义契约	数值全绿但语义错误的 kernel	检查编译产物含所声明语义必需的原语
答案集隔离	参考解泄漏进 agent 上下文	沙盒拷贝白名单强制排除 <code>evaluation/</code>
trajectory 验证	skill 摆着没用、或 no_skill 组偷读 skill	解析 Claude Code 自身的 session transcript

3.2 运行环境

项	配置
GPU 集群	AutoDL 云实例, NVIDIA RTX 5090 32GB, Blackwell sm_120
软件栈	CUDA 12.8, torch 2.8.0+cu128, triton 3.4.0, ninetoothed 0.25.0, Python 3.12
Agent	Claude Code CLI 2.1.206, headless 模式在沙盒目录内运行
模型接入	GLM-5.2 经智谱 Anthropic 兼容端点; DeepSeek-v4-pro[1m] 经 DeepSeek Anthropic 兼容端点。切换后端只改环境变量, 评测代码零改动
资源约束	单 episode 超时 1200 秒; 每个 episode 独立沙盒目录, 互不复用

沙盒是文件级隔离而非容器级。这个已知弱点在实测中真实暴露过一次，见第六节；容器级隔离列入维护计划。

四、自测算子任务与 correctness 明细

4.1 四个自测任务

tests/self_test_tasks.md 定义四个任务对应隐藏任务的四个家族，各有一份完整参考实现：

#	任务	家族	覆盖点	参考实现
1	masked elementwise add	elementwise	广播、bool mask、other= 填充、非 2 幂 shape	examples/01
2	参数化 mean reduction	reduction	reduction 三模式、fp32 累加、NaN/Inf 边界、benchmark	examples/02
3	pixel_unshuffle	layout	缩放因子 2/3/4、contiguous 与非连续、ravel/permute	examples/03
4	softmax 性能回退诊断	perf-diag	生成源码检视、前后 benchmark、最小修复	examples/04

run_self_tests.sh 对四个参考实现做回归检查。要说明白的是：它两遍跑的是同一份参考代码，验证的是示例不腐坏，不是 with/without-skill 对比。真正的 A/B 证据由 harness 以全新 agent 会话产生，见第七节。

4.2 算子 correctness 明细，GLM-5.2 组

表中 matrix 一列是防篡改 oracle 加 MERE/MARE 矩阵的通过比例。聚合行数字逐字来自一手运行日志 cc_glm52_repo.log；per-task 行复原自运行期会话记录，与日志聚合完全自治。

任务	内容	no_skill 总分 / matrix	v0 总分 / matrix	delta
ew01	一维逐元素 add, fp32/fp16	9 / 1.0	9 / 1.0	0
ew02	乘法广播 (M,1)×(1,N)	9 / 1.0	9 / 1.0	0
ew03	ReLU	4 / 0.0	9 / 1.0	+5
ew04	GELU tanh 近似, fp16 数值	4 / 0.0	9 / 1.0	+5
ew05 *	SiLU	4 / 0.0	9 / 1.0	+5
ew06 *	带掩码加法, mask 广播	3 / 0.0	9 / 1.0	+6
合计		33, 2/6 过	54, 6/6 过	+21

带 * 的是 holdout 任务，skill 文本中从未出现过的题目，v0 同样全过，不是背答案。

失败模式值得注意：no_skill 的四个失败全部是 completion 得 0——完全写不出能通过矩阵的 DSL kernel，而不是差容差。GLM-5.2 在无引导时无法稳定掌握 arrangement 与 application 范式；skill 提供家族 playbook 和可运行的调试脚本后，六题全部满分，零负迁移。

4.3 算子 correctness 明细，DeepSeek-v4-pro 组

per-task 数字直接来自素材包内的 ab_operators.csv：

任务	no_skill 总分 / matrix	v0 总分 / matrix	delta	备注
ew01	9 / 1.0	9 / 1.0	0	双方全过
ew02	9 / 1.0	9 / 1.0	0	双方全过
ew03	4 / 0.0	4 / 0.0	0	双方失败，对该模型的真实难题
ew04	4 / 0.0	6 / 0.5	+2	v0 过半矩阵
ew05 *	4 / 0.0	9 / 1.0	+5	v0 独过
ew06 *	9 / 1.0	5 / 1.0, 复核修正 9	0, 修正后	评分器漏检，见下
合计	39, 3/6	42, 修正后 46, 4/6	+7, 修正后	

ew06 的 -4 经人工复核确认为评分器误判，不是负迁移。该 episode 矩阵 2/2 全过，产物 `test_contrib_ew06.py` 是标准的 NineToothed 实现——`import ninetoothed`、`ninetoothed.make(arrangement, application, ...)`，mask 广播用 `tile((1, BLOCK_SIZE_N)).expand(...)` 处理得干净利落，文件在素材包内可查。评分器输出自相矛盾：一边给矩阵判了通过，一边报告“未产出 python 文件”——根因是变更检测在该 episode 漏检了新建文件，导致依赖文件清单的合法性门与风格分连带误判。按产物证据修正后 ew06 为 9 比 9 平局，算子组合计修正为 39 比 46，v0 领先 7 分，零负迁移。表格保留评分器原始输出，正式结论采用修正值；这一漏检已作为评分器缺陷记录进 6.2 节的修复清单。

4.4 诊断任务 correctness

任务	内容	GLM ns→v0	DeepSeek ns→v0
pd01	fp16 softmax NaN 定位修复	5→5	5→5
pd02	mean reduction fp16 精度	3→5	6→6
pd03	add kernel benchmark 设计	3→4	作废 †
pd04	生成源码 tile 与并行参数报告	6→6	5→6
pd05 *	非连续输入错误定位	2→2	2→3
pd06 *	AOT 构建失败定位	5→2	2→5
合计		24→24, 6 题	20→25, 有效 5 题

† DeepSeek 组首轮 pd03 的 no_skill episode 被发现越界读取了残留工作区里的 skill 文件，属于基线数据缺陷。该题双模式作废，不计入正式对比；沙盒清理后的干净重跑随素材包归档。事件经过见第六节。

诊断组的解读：GLM 组打平；DeepSeek 组在有效 5 题上 v0 小幅领先，领先项集中在 pd06 这类需要按既定排查流程走的任务。诊断瓶颈整体上更多在模型的通用排障能力而非领域知识，与中期预判一致。

五、算子性能：benchmark 设计、规模、结果与回退分析

5.1 设计与规模

bench_compare.py 用 CUDA event 计时，warmup 后多次迭代取均值，对比 PyTorch 同语义基线，计算有效带宽与算术强度，按 Roofline 判 compute/memory-bound。自测计划要求任务 2 和 4 携带 benchmark，每个至少 3 个输入规模并给出 bound 结论，与评分门的要求一致。

5.2 实测结果，RTX 5090，fp16

以下均为 2026-07-12 在上述运行环境复测的数据。

任务 2，行求和 reduction，4096 行 × N 列：

N	torch ms / GB/s	nt_sum ms / GB/s	nt/torch
1024	0.0121 / 695	0.0474 / 177	0.25x
4096	0.0174 / 1927	0.0470 / 714	0.37x
4095, 非 2 幂	0.0174 / 1931	0.0429 / 783	0.41x

Roofline 判类：算术强度 0.5 FLOP/byte，远低于 ridge point，属 memory-bound，结论正确。如实说：正确性满分的 nt reduction 尚未达到 PyTorch 级带宽利用，tile 形状还有调优空间。这正是 perf-diag workflow 存在的意义——skill 的价值主张是"正确且可诊断"，不承诺首写即最快。另注：bench_compare.py 的 ridge-point 常数表目前用的 H100 参数，在 5090 上 bound 判类不受影响，常数表更新列入维护计划。

任务 4，softmax 回退修复：

N	torch	nt_slow 回退版	nt_fast 修复版	修复增益	vs torch
1024	0.0324 ms / 518 GB/s	0.0343 / 489	0.0276 / 608	+24%	1.17×
2048	0.0222 / 1514	0.0345 / 971	0.0329 / 1021	+5%	0.67×
4096	0.0480 / 1398	0.0340 / 1972	0.0336 / 1997	+1%	1.43×
4095	0.0484 / 1385	0.0484 / 1385	0.0457 / 1469	+6%	1.06×

5.3 回退分析链路

examples/04 完整演示 perf-diag workflow: nt_slow 的 BLOCK_SIZE 固定为小值, masked lane 浪费带宽; 用 inspect_generated_source.py 读出生成源码中的 tile 配置差异, 判定瓶颈在无效访存; 最小修复是把 BLOCK_SIZE 设为行长; 复测确认. N=1024 时修复带来 24% 带宽提升, N=4096 时修复版超 PyTorch 43%. 这条"测、读生成源码、定位、最小改、复测"的链路就是 pd 家族任务考察的能力, 也是 A/B 中 agent 被引导执行的流程。

5.4 A/B episode 中 agent 产出的 benchmark

GLM 组 pd03 的 v0 episode 产出合格 benchmark 工件拿到 perf 子分, 对应 no_skill 为 0。DeepSeek 组首轮 pd03 v0 也产出了结构完整的 bench_add.csv, 8 个规模点、双 dtype、均值与标准差、带宽换算、bound 结论俱全, 但其 NineToothed 侧计时疑似把 Triton JIT 首次编译计入了测量, 绝对数值不可信, 本报告不引用。给 benchmark 数值加可信性门——把 robust_bench.py 的离群剔除和异常加速比拦截接进评分——列在维护计划第一位。

六、失败诊断案例与修复

6.1 Agent 侧案例: fp16 softmax NaN

DeepSeek 组 pd01 的 with-skill episode 按工作流产出了完整的 diagnosis.md, 原文在素材包内。要点:

- 根因一, 缺 subtract-max 稳定化。fp16 下 exp(x) 在 x>11.09 即溢出为 Inf, Inf/Inf 得 NaN; 输入放大十倍时约 14% 元素越界。
- 根因二, fp16 累加。数百项求和时约 3.3 位十进制精度饱和, 概率整体偏移, 对应 common-errors.md 里"allclose 挂但 cosine 接近 1"的症状条目。
- 最小修复两行: 先减行最大值, exp 与求和链路上转 fp32, 最后 cast 回输出 dtype。arrangement 与接口零改动。
- 验证闭环: 给出修复前后生成源码 diff, 多出 tl.max 与 tl.cast, 附矩阵复测命令。

该 episode 的 transcript 显示 agent 真实调用了 Skill 工具并读取 perf-diag 与 common-errors 两份 reference, 诊断路径与 skill 的症状表一致。

6.2 评测体系的自我发现与修复

评测体系本身也按"症状、根因、最小修复、复验"的循环迭代过七轮，全部留档。前五例是开发期修复：agent 模仿仓库真实惯例用 `@ninetoothed.jit` 反被判未用 DSL，根因是检查器只认 `make()`，修复后两种合法写法都识别；repo-aware 模式全树扫描把框架源码本体当成 agent 产出，修复为只对真实 diff 打分；合法性扫描误伤测试文件里合法的 torch 参考计算，修复为只扫 solution 文件；`run_self_tests.sh` 曾自称 A/B，重标为回归检查；包内曾有一个依赖从未产出数据的 `strategy_selector.py`，作为广告功能移除。

第六例是评分器变更检测的漏检。DeepSeek 组 ew06 的 with-skill episode 产出了完全合规的 NineToothed 实现且矩阵全过，但变更检测漏掉了新建文件，连带触发合法性门误判罚。报告复核阶段通过工作区产物与评分器输出的自相矛盾定位到该缺陷，按证据修正了该题结论，检测逻辑修复列入维护计划。

第七例发生在终期实测中，值得完整记录。DeepSeek 组首轮跑完后，trajectory 验证的全量扫描发现 pd03 的 no_skill episode 有两次越界 Read——它在探索文件系统时发现了此前一次运行残留的工作区，里面装着完整的 skill，读走了两个 benchmark 示例文件，然后拿下 no_skill 组诊断最高分。这是典型的基线污染。处理方式：pd03 双模式作废，不进正式对比；备份后清理宿主机上的残留工作区；用一致的代码和干净沙盒重跑该题，重跑输出随素材包归档备查。重跑结果印证了污染判断：失去越界读到的 skill 示例后，干净基线从 6 分回落到 2 分，with-skill 侧 3 分，方向与其余诊断题一致。两点补充：其一，污染方向是抬高基线、压低 skill 的表观增益，即便不处理也不会夸大本报告的结论；其二，发现它的正是本项目自建的 trajectory 验证——评测体系具备发现自身缺陷的能力，这比一份表面干净的数据更有说服力。

6.3 归因机制

`failure_classifier.py` 的四类归因同时服务两端。agent 运行期，终止规则强制失败后先分类，杜绝无效重试；进化路线里，`guidance_error` 定位到该改的 reference 文件，`code_error` 不触发 skill 编辑，保证进化只在 skill 文本真有缺口时动它。

七、与不用 skill 的 AI 基线对比

7.1 实验设置

为检验 skill 对不同能力档位基座的泛化性，选取两个基座各跑一遍完整 A/B：GLM-5.2 代表当前主流国产基座档位，DeepSeek-v4-pro[1m] 代表前沿编码基座档位。两组共用同一套任务、沙盒、评分器，唯一差异是模型端点。

固定项	取值
Agent	Claude Code CLI, headless, 沙盒内运行
任务	evaluation/proxy_tasks/manifest.json , elementwise 6 题 + perf_diag 6 题
沙盒	repo-aware: 完整 ninetoothed 仓库副本, 交付物为 tests/test_contrib_<id>.py
唯一变量	v0 组多一段"skill 已安装, 请使用"提示, 且 skill 物理装入 .claude/skills/
评分	同一 rubric、同一防篡改 oracle、同一合法性门, 两组一视同仁
硬件	见 3.2 节

7.2 汇总结果

组	家族	no_skill	v0	delta
GLM-5.2, 24 episodes	算子 ×6	33 分, 2/6 过	54 分, 6/6 过	+21 分
	诊断 ×6	24 分, 3/6	24 分, 3/6	0
DeepSeek-v4-pro, 24 episodes †	算子 ×6	39 分, 3/6	46 分, 4/6	+7
	诊断, 有效 5 题	20 分	25 分	+5

† 算子组 v0 为 ew06 复核修正值, 评分器原始输出 42 分, 修正依据见 4.3 节; pd03 双模式因基线数据缺陷作废, 见 6.2 节。GLM 组消耗约 2200 万 tokens, DeepSeek 组约 3300 万 tokens、35.7 美元。

7.3 跨基座解读

GLM-5.2 组的结果是决定性的: 四个 no_skill 完全写不出的算子, 包括两个 holdout 题, with-skill 全部满分, 零负迁移。skill 的增量集中在"正确写出 DSL 算子"这一硬能力上。

DeepSeek-v4-pro 组同样是净正增益: 算子组修正后 +7 分、多过一题 holdout, 诊断有效题 +5 分。这个基座在 SWE-bench Verified 上约 80%, 无 skill 也能解出相当一部分任务, 增益幅度小于 GLM 组, 符合 SkillsBench 报告的"skill 增益随基座能力递减"规律, 但方向一致。经复核, 两个基座共 46 集有效对比中没有一例确证的负迁移。

两组组合起来的结论: 本 skill 对 GLM 档位的基座价值最大, 直接决定任务成败; 对前沿基座, 增益转向 holdout 难题、过程正确性与可验证工件产出。用两个档位的基座做同一套测试, 本身也验证了 skill 与评测链路的可移植性: 换后端只改环境变量。

7.4 Trajectory 证据

对 DeepSeek 组全部 episode 的 Claude Code session transcript 离线扫描, 机制已接入 journal 常态记录:

组	Skill 工具正式调用	读取 skill 文件	零接触
v0 ×12	10/12	12/12	0
no_skill ×12	0	首轮 1 例越界, 已作废重跑	11/12

典型 v0 轨迹：调用 Skill 工具，读 references，模仿 examples，运行 debug_arrangement 与 run_correctness_matrix，最后读生成源码缓存做检视——与 SKILL.md 工作流逐步吻合。"skill 被真实使用"从假设变成了可审计的断言。

7.5 过程级分析：自测通过不等于真正确

agent 在 episode 内部是多轮迭代的：写 kernel、跑测试、失败、修、再跑。逐 transcript 统计每个算子 episode 对自己交付物测试的运行轮次与首次通过轮次，再对照独立 oracle 的最终判定，暴露出一个二值通过率完全看不到的现象：

组	自测最终通过	独立 oracle 通过	假自信案例
GLM no_skill x6	6/6	2/6	4
GLM v0 x6	6/6	6/6	0
DeepSeek no_skill x6	6/6	3/6	3
DeepSeek v0 x6	6/6	5/6 全过或过半	1

no_skill 组的 agent 几乎每题都在若干轮内让自己写的测试转绿，然后自信收工——但 GLM 组有四题、DeepSeek 组有三题没能通过独立 oracle 的完整矩阵。根因是验证不充分：自写测试只覆盖了自己假设的 shape 与条件，比如把逐元素任务默认成一维输入，而 oracle 按任务 spec 的完整 shape×dtype 矩阵测。with-skill 组的差别不在"写得更快"——GLM 组两边首次通过轮次相当，都在 1 到 3 轮上下——而在验证得更对：skill 工作流强制先用 gen_pytorch_oracle.py 生成参考对照、再跑满矩阵，自测绿灯与独立判定在 GLM 组 12 集上完全一致。

换句话说，这个 skill 消除的主要不是编码失误，而是假自信：让 agent 的"我做完了"与评委的"确实做对了"对齐。对自动化协作场景，这比单点通过率更有工程价值——一个会报假成功的 agent 比一个报真失败的 agent 危险得多。

7.6 自测计划达标线判定

达标线	GLM 组	DeepSeek 组
每题 v0 ≥ no_skill	达标，12/12	达标，复核修正后 11/11 有效题
至少 2 题有效性能验证	达标	达标
0 负迁移	达标	达标，复核修正后

7.7 覆盖范围与局限

已覆盖 elementwise 与 perf_diag 两家族、两种模式、两个基座，共 48 个 episode，其中 pd03 双模式作废后正式对比按 46 集计。reduction 与 layout 家族已有任务定义和参考解，受 GPU 预算限制未跑 A/B，列入维护计划。每个 cell 只有一次采样，±1 分的波动没有统计意义，本报告的结论只建立在方向一致的大差距和结构性证据上。GLM 组的 per-task 行为会话记录复原，聚合行有一手日志，已在 4.2 节标注。

八、Stage 3 自进化：实测、局限与归因

8.1 两条路线与实测结果

中期规划了两条自进化路线。版本 B 是轻量 MOO：评估当前 skill、归因失败、生成 prune/substitute/reorder 编辑提案、过"通过率保持门"后晋升。版本 A 是重控制器：contextual bandit 选策略、三层记忆积累经验、高置信规则蒸馏回 skill 文本。

版本 B 在 elementwise 切片上跑了四轮独立进化，每轮三代，覆盖 Qwen 与 GLM 两种后端。真实的代际记录如下：

轮次	起始 fitness	gen1	gen2	gen3
第 1 轮	16	reorder 接受, 16→21	substitute 接受, 持平	无提案
第 2 轮	16	substitute 接受, 持平	无提案	无提案
第 3 轮	21	substitute 接受, 持平	无提案	无提案
第 4 轮	21	substitute 接受, 持平	无提案	无提案

"无提案"的原始记录是 no guidance-error proposals (all code_error/unknown) ——归因器认定剩余失败都不是 skill 文本能修的。第 1 轮那次 +5 也当不得真增益：同一份 v0 在不同轮次的起始 fitness 就在 16 到 21 之间波动，单次改进落在评估噪声带内。版本 A 跑通了全链路但从未蒸馏出高置信规则，成本又数倍于轻量路线，放弃并从代码库移除。

四轮的共同形态值得注意：主导终态不是"守门员拒绝了坏编辑"，而是上游归因饿死——循环还想继续，但没有东西可编辑了。下面四个归因按证据强度排列。

8.2 归因一：学习信号贫瘠，轨迹不可见

归因器的全部输入只有两样：episode 的最终错误文本和一个十分制分数。它看不到 agent 的执行轨迹，于是无法区分三种根本不同的失败：指导写错了（改文本有用）、指导没被读（改了也白改）、模型能力不够（谁改都没用）。区分不了就只能保守，保守的结果就是把绝大多数失败归入 code_error 和 unknown，编辑提案枯竭。这不是归因器实现得差——黑盒外环拿不到做对归因所需的信息。

有一个反证让这条归因格外扎实：项目末期为验证目的开发的 trajectory 解析证明，轨迹其实一直可得——Claude Code 把逐工具调用记录写在本地 transcript 里，我们靠解析它抓到了基线污染和评分器漏检两个真问题。但进化循环成型在先，从未消费这份数据。如果重来，第一件事就是把轨迹特征喂给归因器：目标章节是否真被读过、失败发生在第几步、哪些脚本跑了没跑。guidance_error 的判定会从"按症状猜"变成"按观测判"。

再往前一步是自研或深度插桩的 agent 框架。它带来的不只是观测，而是 step 级重放：从失败步之前的状态换上编辑后的 skill 文本继续执行，单次评估成本从一整个 episode 降到几步，credit assignment 从"整包文本对总分"细化到"具体段落对具体步骤"。黑盒 CLI 有一体两面：作为被测对象它忠实于真实用户场景，作为进化环境它信息贫瘠、只可观测不可干预。本项目选它是为了让 A/B 结论可信，代价就由 Stage 3 承担了。

8.3 归因二：编辑算子的两难

实测观察到 LLM drafter 用泛泛的正确性套话覆盖了原有的精确 pitfall——一条 rank/tile 修复要点被冲掉，此后编辑策略改为只追加。但追加式扩写与 SkillMOO 的结论以及我们自己的数据一致：不产生增益。真正可能有用的 prune 和 reorder 又依赖精确归因，回到 8.2 的死结。一句话概括：无轨迹的黑盒进化里，安全的编辑没用，有用的编辑不安全。

8.4 归因三：基线位置与评估经济学

v0 是对照真实源码逐条核验的手工基线，在评测切片上已接近该模型能力下的局部最优——剩余失败经归因多为模型能力上限而非文本缺口，这从 no_skill 与 v0 在难题上同时失败也能交叉印证。同时，同一 v0 的起始 fitness 在轮次间波动达 5 分，说明单次评估噪声与可期望的最大增益同量级；要在这种信噪比下统计确认一次真改进，每代需要多次重复评估，而每次评估都是真实 token 与 GPU 开销。进化循环不是没找到改进，而是在这个噪声水平和预算约束下，可确认的改进空间本来就窄。

8.5 归因四：自进化产物与开源交付形态的张力

最后一层不是技术问题。交付物是开源仓库里接受人审的 curated 包，核心纪律是每条承重结论可溯源——references 里的 [E]/[S]/[I] 标签就是这条纪律的落地。机器改写的段落天然只有推断级证据，直接合入会稀释整包的可审计性；迭代自动产生的 diff 也不匹配 PR 的人审节奏。所以即便进化循环完全成功，它的正确位置也是 harness 侧的提案生成器：机器提案，人工核验并升级证据等级，然后才进包。promote 机制实际正是按此设计的——在隔离副本上逐代打 git tag，从不直接写回交付分支。

8.6 小结

自进化在本项目失效的不是理念，是三个前提一个都不满足：轨迹可见、编辑可靠、评估可负担。按性价比排序的解锁路径：轨迹特征进归因最便宜，transcript 已经可得，零额外基建；step 级重放次之，需要自研或深插桩框架；每代重复采样降噪最贵，纯粹是预算问题。在前提补齐之前，手工 curation 加独立 A/B 验证仍是把 skill 做对的最短路径——这也是本项目最终交付所走的路。

九、安全、依赖、授权与引用披露

9.1 安全与合规

无 API 密钥、无网络调用、无隐藏答案、不删测试。脚本只读或以 list 形式 argv 起子进程，`shell=False`，无 eval、exec、动态 import、shell 拼接。安装即文件发现，不执行任何代码。`evaluation/proxy_tasks/` 的参考解仅用于打分，沙盒拷贝白名单强制排除，任何时候不进 agent 上下文；这是选手自建的自测集，与官方隐藏评测无关。

9.2 依赖

ninetoothed $\geq$ 0.25.0, triton $\geq$ 3.0.0, torch $\geq$ 2.4.0, pytest $\geq$ 7, CUDA 12.x。无 CUDA 时 bench 降级为墙钟计时并告警，GPU 测试自动跳过。numpy 与 sympy 随 ninetoothed 传递安装。

9.3 引用

以下均为方法论借鉴，无代码复制，完整披露见 `REFERENCE.md`：

- SkillsBench, arXiv 2602.12670: 人工整理优于自生成的结论，以及本报告 7.3 节的天花板解读框架。
- SkillMOO, arXiv 2604.09297: prune/substitute 有效而 expansion 无效，指导进化路线的编辑算子选择。
- Anthropic Agent Skills 文档：包结构、渐进披露、脚本承载确定性。
- Ascend agent-skills: Code Repair 与 Prompt Repair 的二分，`failure_classifier` 归因设计的出发点；`dsl_limit` 终态类为本项目自有扩展。
- KernelSwift, 上海 AI Lab: 三技术 reward-hacking 检测，对应 `reward_hacking_guard` 与 `robust_bench` 的测量协议；生成源码语义契约检查为本项目自有扩展。
- KernelBench, arXiv 2502.10517: 编译、正确、合法三门评分惯例，对应 `verifier_spec.md` 的三门映射。
- MusaCoder, arXiv 2606.04847: banned fallback 命中即零分，对应 `banned_fallback_analysis` 及其接线。
- NineToothed 本体, InfiniTensor/ninetoothed, Apache-2.0: 全部 API 对照 0.25.0 源码核验，示例风格取自其自带测试。
- 第十节引用的行业事件与数据——Sakana AI CUDA Engineer 撤回事件、华为 Kernel Foundry 对 KernelLLM 的剔弊对照、KernelBenchX 第三方评测、AWS NKI-Agent 消融——均出自公开论文与公开报道，仅用于领域定位，不构成本项目的实验结论。

9.4 生成式 AI 辅助披露

本 skill 的文本、脚本与配套评测框架由 Anthropic Claude 在 Claude Code 环境中、作者全程指导下辅助起草，作者逐项审核，API 断言对照 ninetoothed 0.25.0 源码核验，references 以 [E]/[S]/[I] 标注证据等级。A/B 评测中的被测 agent 为 Claude Code CLI，分别接 GLM-5.2 与 DeepSeek-v4-pro[1m]，开发期另使用过本地 Qwen2.5-Coder-14B；这些模型的输出仅作为评测数据，不构成 skill 内容。本报告由 Claude 辅助撰写，全部数字出自真实运行工件，作者审核定稿。

十、后续演进路线

这个项目值得继续的不是修一列小问题，而是把已验证的三样东西——可信的测量、curated 的知识载体、可复现的对比方法——放到 kernel 生成这个正在快速演化的领域里做大。以下按基石、扩展、定位、解锁四层展开。

基石：让测量护城河跟上领域的对抗速度。 LLM 生成 kernel 这个方向过去两年最重要的教训不是哪家模型更强，而是评测被反复攻破：Sakana AI 的 CUDA Engineer 从宣称百倍加速到 48 小时内撤回；华为对 KernelLLM 做剔弊对照后 correctness 从 55% 掉到 19%；KernelBench 为此做了整轮反作弊改造；第三方基准 KernelBenchX 把两个自报接近满分的系统分别打回 10.8% 与 30.7%。本项目在小尺度上独立撞见并处置了同一类问题——防篡改 oracle、合法性门、语义契约、trajectory 验证，以及靠后者当场抓到的基线污染。因此测量侧的后续投入不是杂项维护，而是这套体系的核心资产：robust_bench 的离群剔除与异常加速比拦截接入评分门、每 cell 重复采样给出方差、episode 容器级隔离、变更检测与 Roofline 常数表跟进新硬件。在一个“90% correctness 不能直接读成语义正确”的领域，经得起第三方复跑的结论本身就是稀缺品。

扩展：从 12 题 A/B 到 NineToothed 的公共基准。 补全 reduction 与 layout 家族只是第一步。24 题任务集加防作弊 harness 加 trajectory 验证，这套组合有条件长成 NineToothed 生态的公共评测基准：领域内 backward 与 training kernel 评测几乎是全空白，DSL 层的 kernel-agent 基准也没有现成标准。路径是把任务集向 matmul、attention、backward 家族扩展，把 harness 的一条命令复现固化进 CI，让“新模型来了跑一遍、新 DSL 版本发了跑一遍”成为常态回归——从参赛自测长成社区基础设施。

定位：skill 文本是领域决策的第三种载体。 当前领域里“下一步优化什么”的决策有两种载体在竞争：蒸进 RL 权重（字节 CUDA Agent、MusaCoder 等路线），或者靠强模型加工具在上下文里现推（NVIDIA、AWS 的多轮系统），两条路线的公开证据互相矛盾、尚无定论——AWS 甚至报告了 RL 微调的负结果，而其单次生成到 agent 循环的跃升（6% 到 77%）远大于任何训练侧增量。这与本项目的观察一致：增益主要来自脚手架与知识载体，不是权重。curated skill 文本是第三种载体：零训练成本、逐条可审计、随 DSL 版本一起演进，并且与前两条不互斥。落到维护动作上，[S]/[I] 标签条目随 ninetoothed 版本升级的复核流程，本质是给知识载体建 CI；common-errors 按 unknown 归因回填是它的增量学习。再往上游走一步：skill 的家族 playbook 可以反哺仓库自身的 CONTRIBUTING 与开发文档，从参赛工件转正为项目的贡献者引导资产。

解锁：自进化的三个前提，按性价比执行。 第八节的归因给出了顺序：先做轨迹特征进归因——transcript 已经落盘，零额外基建，就能把 guidance_error 判定从按症状猜变成按观测判；再评估 step 级重放框架——需要自研或深插桩 agent 环境，与领域内 profiler 引导的多轮系统方向一致；评估预算翻倍降噪放最后，纯粹是钱的问题。在前提补齐之前，进化循环保持 harness 侧提案生成器的定位，机器提案、人工核验、证据升级后进包。

复现入口始终是一条命令：

```
SOLVER=claude SKILL_ROOT=<skill> REPO_ROOT=<ninetoothed> TAG=<tag> bash run_comparison.sh
```

附录：数据素材索引

全部原始工件归档于 `final-report-artifacts/`：

素材	路径
GLM-5.2 A/B 一手日志	<code>2026-07-10-glm-qwen14b-earlier-session/logs/cc_glm52_repo.log</code>
DeepSeek 24 集完整 journal 与 per-task CSV 及 episode 工作区	<code>2026-07-12-deepseek-v4pro/deepseek-v4pro-newarch-COMLETE-24of24/</code>
pd03 干净重跑输出，归档备查不入正式对比	<code>2026-07-12-deepseek-v4pro/pd03-rerun/</code>
全部 Claude Code session transcript，逐 tool 调用轨迹	<code>2026-07-12-deepseek-v4pro/claude-session-transcripts/</code>
skill_engagement 端到端验证样本	<code>2026-07-12-deepseek-v4pro/engagement-test.jsonl</code>
RTX 5090 复测 benchmark	<code>2026-07-12-deepseek-v4pro/bench-rerun-5090/</code>
基线污染源工作区备份	<code>2026-07-12-deepseek-v4pro/stale-workspace-contamination-source/</code>
pd01 诊断原文	<code>.../ep_diag/episodes/baseline_diag/gen0/pd01_v0/diagnosis.md</code>
GLM 与 Qwen-14B 早期全部 journal	<code>2026-07-10-glm-qwen14b-earlier-session/nt-run/</code>

每条 journal 记录含六子项分数、总分、token 成本、skill_engagement 及 transcript 路径，任何结论可回溯到具体 episode 的原始轨迹。